GUIDE PRATIQUE

Git, GitHub et intégration continue (CI)

Comprendre les concepts, maîtriser les commandes et travailler comme en équipe.

OBJECTIF DU SUPPORT
Apprendre à versionner un projet, collaborer avec GitHub, travailler avec des branches et des Pull Requests, corriger des erreurs courantes et automatiser des vérifications avec GitHub Actions.

Parcours progressif • Exemples commentés • Exercices pratiques

Support de formation DevOps — édition pédagogique

Comment utiliser ce document

Ce cours suit un parcours pratique : on commence par les notions essentielles, puis on met en place un dépôt local et distant, on apprend à collaborer avec des branches, et on termine par une première chaîne d'intégration continue (CI). Les commandes sont illustrées sur un projet d'apprentissage.

- Exécute les commandes dans ton terminal, une étape à la fois.
- Lis les sorties de git status, git log et des commandes GitHub : elles permettent de comprendre l'état réel du dépôt.
- Les exemples utilisent un dépôt fictif nommé devops-formations. Adapte les chemins et l'URL à ton projet.
- Avant une commande destructive, vérifie toujours ce qu'elle va modifier.

Règle d'or
Git enregistre l'historique du projet. GitHub héberge le dépôt distant et ajoute des outils de collaboration. GitHub Actions automatise des tâches, notamment les tests et contrôles CI.

Sommaire

- 1. Git, GitHub et CI : les différences
- 2. Installer et configurer Git
- 3. Comprendre le fonctionnement de Git
- 4. Créer un dépôt et réaliser ses premiers commits
- 5. Relier Git au dépôt GitHub
- 6. Lire l'historique et comparer les changements
- 7. Branches, fusion et Pull Requests
- 8. Annuler ou récupérer des changements
- 9. Résoudre un conflit de fusion
- 10. GitHub Actions et intégration continue
- 11. Bonnes pratiques de travail
- 12. Dépannage : erreurs fréquentes
- 13. Exercices pratiques et corrigés
- 14. Fiche mémo des commandes

1. Git, GitHub et CI : les différences

1.1 Git

Git est un système de contrôle de version distribué. Il conserve l'historique des modifications d'un projet et permet de retrouver des versions précédentes, de créer des branches et de fusionner des travaux.

1.2 GitHub

GitHub est une plateforme en ligne qui héberge des dépôts Git. Elle facilite le partage du code, les Pull Requests, les revues de code, le suivi des problèmes et l'automatisation avec GitHub Actions.

1.3 CI — Intégration continue

L'intégration continue (Continuous Integration, CI) consiste à vérifier automatiquement les changements du projet, par exemple à chaque push ou à chaque Pull Request. Une CI peut contrôler la présence de fichiers, compiler le projet et lancer des tests.

Outil / notion	Rôle	Exemple
Git	Versionner localement le projet	git add, git commit, git log
GitHub	Héberger et collaborer	push, Pull Request, revue de code
GitHub Actions / CI	Automatiser les contrôles	workflow lancé sur push ou pull_request

À retenir
Git fonctionne même sans connexion à GitHub. GitHub ajoute un espace distant et des outils d'équipe. La CI automatise des vérifications reproductibles.

2. Installer et configurer Git

2.1 Vérifier l'installation

```
git --version
```

Si une version de Git est affichée, Git est disponible dans le terminal. Sur macOS, Git peut être installé avec les outils de ligne de commande Xcode ou via un gestionnaire de paquets.

2.2 Définir son identité

```
git config --global user.name "Prenom Nom"
git config --global user.email "adresse@example.com"
git config --global --list
```

Le nom et l'adresse e-mail sont inscrits dans les métadonnées des commits. Choisis une adresse adaptée à ton compte GitHub.

2.3 Configuration locale ou globale

- Configuration globale : s'applique par défaut à tous les dépôts de l'utilisateur.
- Configuration locale : s'applique uniquement au dépôt courant et peut remplacer la configuration globale.

```
git config user.name "Nom pour ce projet"
```

Sécurité
Ne place jamais de mot de passe, clé privée, jeton d'accès ou fichier .env contenant des secrets dans un dépôt. Si un secret est commité, le supprimer du dernier commit ne suffit pas toujours : il faut le révoquer/renouveler.

3. Comprendre le fonctionnement de Git

Git travaille avec plusieurs zones. Comprendre ces zones évite la plupart des erreurs de débutant.

Zone	Signification	Commande utile
Working tree	Fichiers sur lesquels tu travailles	git status

Staging area / index	Changements sélectionnés pour le prochain commit	git add
Repository local	Historique des commits enregistré localement	git commit, git log
Repository distant	Copie hébergée sur GitHub	git push, git fetch, git pull

Le cycle classique

```
modifier un fichier
git status
git add README.md
git commit -m "docs: mettre a jour le README"
git push
```

Le commit enregistre un instantané des changements sélectionnés. Le push transmet les commits locaux au dépôt distant, si la branche et les droits sont correctement configurés.

4. Créer un dépôt et réaliser ses premiers commits

4.1 Initialiser un dépôt

```
mkdir -p ~/Documents/formations/devops-formations
cd ~/Documents/formations/devops-formations
git init -b main
```

Cette commande initialise un dépôt et crée la branche initiale main. Si ta version de Git ne prend pas en charge -b, utilise git init puis git branch -M main.

4.2 Ajouter un README

```
echo "# Formation DevOps" > README.md
git status
git add README.md
git status
git commit -m "docs: ajouter le README"
```

Les deux appels à git status permettent de comparer l'état avant et après la sélection du fichier. Après le commit, git status devrait indiquer que l'arbre de travail est propre si aucun autre fichier n'a changé.

4.3 Deux exemples importants : add et commit

Exemple A — sélectionner un fichier précis :

```
git add README.md
git commit -m "docs: expliquer les objectifs de la formation"
```

Exemple B — sélectionner plusieurs fichiers connus :

```
git add README.md Dockerfile .github/workflows/ci.yml
git commit -m "build: ajouter les bases de la CI"
```

Bonne pratique

Préférer git add fichier à git add . quand tu veux garder un contrôle précis sur ce qui entrera dans le prochain commit. Vérifie le contenu avec git diff --staged avant de valider.

5. Relier Git au dépôt GitHub

5.1 Créer le dépôt distant

1. Sur GitHub, crée un nouveau dépôt.
2. Si le projet existe déjà en local, évite de générer un README distant en double lors de la création, sauf si tu sais comment fusionner les historiques.
3. Copie l'URL du dépôt distant HTTPS ou SSH.

5.2 Ajouter origin et publier main

```
git remote add origin https://github.com/UTILISATEUR/devops-formations.git
git remote -v
git push -u origin main
```

origin est le nom conventionnel du dépôt distant. L'option -u associe la branche locale main à origin/main pour simplifier les push et pull suivants.

5.3 Deux scénarios de publication

Exemple A — première publication d'une branche :

```
git push -u origin main
```

Exemple B — publier une branche de fonctionnalité :

```
git switch -c feature/ajouter-section-docker
# modifier README.md, puis :
git add README.md
git commit -m "docs: ajouter une section Docker"
git push -u origin feature/ajouter-section-docker
```

5.4 Récupérer un projet existant

```
git clone https://github.com/UTILISATEUR/devops-formations.git
cd devops-formations
git status
```

clone télécharge le dépôt et son historique, puis configure normalement origin. Pour mettre à jour un dépôt déjà cloné, on utilise plutôt fetch/pull.

6. Lire l'historique et comparer les changements

Commande	Utilité
git status	Voir les fichiers modifiés, sélectionnés ou non suivis
git log --oneline	Afficher l'historique sous forme compacte
git log --oneline --graph --all	Visualiser les branches et les fusions

git diff	Voir les changements non sélectionnés
git diff --staged	Voir les changements sélectionnés pour le prochain commit
git show --stat HEAD	Résumé du dernier commit
git remote -v	Voir les dépôts distants configurés

Exemple A — inspecter avant de committer

```
git status
git diff
git diff --staged
```

git diff et git diff --staged examinent des zones différentes. Si git diff n'affiche rien mais qu'un fichier est déjà sélectionné avec git add, utilise git diff --staged.

Exemple B — retrouver un commit

```
git log --oneline --graph --decorate -10
git show <identifiant-du-commit>
```

Remplace le texte entre chevrons par l'identifiant réel du commit. Ne copie pas les chevrons dans la commande.

7. Branches, fusion et Pull Requests

7.1 Pourquoi créer une branche ?

Une branche permet de développer une fonctionnalité ou une correction sans modifier directement la branche principale. Une fois le travail prêt, il peut être intégré après vérification.

```
git branch
git switch -c feature/git-branches
# effectuer une modification
git add README.md
git commit -m "docs: expliquer les branches"
git push -u origin feature/git-branches
```

7.2 Pull Request (PR)

Une Pull Request propose d'intégrer une branche dans une autre, généralement feature/... vers main. Elle permet de comparer les changements, discuter du code, lancer la CI et obtenir une validation avant fusion.

4. Pousser la branche sur GitHub.
5. Ouvrir une Pull Request et indiquer le contexte, les changements et les tests effectués.
6. Vérifier les différences et les contrôles CI.
7. Fusionner si les contrôles et les règles du projet sont satisfaits.
8. Mettre à jour main en local, puis supprimer la branche devenue inutile si approprié.

7.3 Deux méthodes de fusion

Exemple A — fusion locale classique (si le flux de travail du projet l'autorise) :

```
git switch main
```

```
git pull --ff-only origin main
git merge feature/git-branches
git push origin main
```

Exemple B — workflow avec Pull Request :

```
git switch main
git pull --ff-only origin main
git switch -c feature/ajouter-ci
# effectuer les changements et les committer
git push -u origin feature/ajouter-ci
# ouvrir ensuite la Pull Request sur GitHub
```

Important
Dans un projet d'équipe, respecte les règles du dépôt : si main est protégée et que les changements doivent passer par une PR, ne contourne pas cette procédure par un push direct.

8. Annuler ou récupérer des changements

Toutes les commandes d'annulation n'ont pas le même effet. Avant d'agir, vérifie git status et git log --oneline.

Commande	Effet principal	Précaution
git restore fichier	Abandonne les changements non sélectionnés de ce fichier	Les changements concernés sont perdus
git restore --staged fichier	Retire le fichier de la staging area	Conserve les modifications dans le working tree
git commit --amend	Modifie le dernier commit	À éviter sur un commit déjà partagé sans coordination
git revert <commit>	Crée un nouveau commit qui inverse un commit	Adapté à un historique partagé
git reset --soft HEAD~1	Déplace HEAD d'un commit et conserve les changements sélectionnés	Réécrit l'historique local
git reset --mixed HEAD~1	Déplace HEAD et remet les changements dans le working tree	Le commit est retiré de la branche locale

Exemple A — retirer un fichier de la sélection

```
git add README.md
git restore --staged README.md
git status
```

Le fichier reste modifié dans le répertoire de travail ; il n'est simplement plus sélectionné pour le prochain commit.

Exemple B — annuler un commit déjà partagé

```
git log --oneline
git revert <identifiant-du-commit>
git push
```

git revert est généralement plus sûr que reset pour annuler un changement déjà poussé, car il ajoute un commit inverse au lieu de réécrire l'historique partagé.

8.1 Stash : mettre un travail de côté

```
git stash push -m "travail temporaire"
git stash list
git stash pop
```

Le stash est utile pour mettre temporairement de côté des changements locaux non committés. Vérifie le résultat après stash pop, car des conflits peuvent survenir.

8.2 Reset : comprendre les trois modes

Mode	HEAD / commit	Staging area	Fichiers de travail
--soft	Déplacé	Changements conservés	Changements conservés
--mixed (défaut)	Déplacé	Réinitialisée	Changements conservés
--hard	Déplacé	Réinitialisée	Fichiers réinitialisés ; modifications perdues

Attention
N'utilise pas git reset --hard sans vérifier l'état du dépôt et sans être certain de vouloir perdre les modifications locales concernées.

9. Résoudre un conflit de fusion

Un conflit survient lorsque Git ne peut pas déterminer automatiquement comment combiner des changements, par exemple lorsque deux branches modifient les mêmes lignes d'un fichier.

9.1 Procédure générale

9. Lancer la fusion ou mettre à jour sa branche.
10. Exécuter git status pour identifier les fichiers en conflit.
11. Ouvrir chaque fichier et examiner les marqueurs de conflit.
12. Choisir ou combiner les bonnes lignes, puis supprimer tous les marqueurs.
13. Exécuter les tests ou contrôles nécessaires.
14. Ajouter les fichiers résolus et terminer la fusion.

```
git status
# éditer le fichier en conflit, puis :
git add README.md
git commit
```

9.2 Exemple de marqueurs

```
<<<<<<< HEAD
Texte de la branche actuelle
=======
Texte venant de la branche fusionnée
>>>>>>> feature/ma-branche
```

Le bloc entre HEAD et ======= correspond à la version actuelle ; le bloc après ======= correspond à l'autre version. Le texte final peut être l'un, l'autre ou une combinaison.

9.3 Deux façons de terminer

- Si la fusion locale est en cours : après résolution et git add, terminer avec git commit si Git ne l'a pas déjà fait automatiquement.
- Si le conflit survient pendant un rebase : après résolution et git add, utiliser git rebase --continue. Pour abandonner, git rebase --abort.

Avant de commencer
Si tu n'es pas sûr de l'état du dépôt, ne lance pas d'autres commandes au hasard : exécute git status et lis les indications de Git.

10. GitHub Actions et intégration continue

10.1 À quoi sert un workflow ?

GitHub Actions exécute automatiquement des étapes définies dans un fichier YAML situé généralement dans .github/workflows/. Un workflow peut être déclenché lors d'un push, d'une Pull Request ou manuellement.

10.2 Premier workflow de contrôle

Exemple de workflow minimal pour vérifier la présence du README à chaque push et Pull Request :

```
name: CI

on:
  push:
  pull_request:

jobs:
  check-readme:
    runs-on: ubuntu-latest
    steps:
      - name: Recuperer le depot
        uses: actions/checkout@v4

      - name: Verifier le README
        run: test -f README.md
```

Le nom du job et les étapes peuvent être adaptés. Ce workflow ne compile pas le code : il vérifie seulement que le fichier README.md existe.

10.3 Ajouter le workflow au dépôt

```
mkdir -p .github/workflows
# créer le fichier .github/workflows/ci.yml avec le YAML ci-dessus
git status
git add .github/workflows/ci.yml
git commit -m "ci: ajouter un controle du README"
git push
```

10.4 Deux exemples de CI

Exemple A — contrôle simple d'un fichier :

```
run: test -f README.md
```

Si le fichier existe, la commande se termine avec un code de sortie 0 et l'étape réussit. S'il manque, la commande échoue et le job devient rouge.

Exemple B — contrôler plusieurs fichiers indispensables :

```
run: |
  test -f README.md
  test -f pom.xml
  test -d src
```

Ce deuxième exemple convient à un projet Maven qui contient un README, un pom.xml et un dossier src. Pour un dépôt qui n'est pas un projet Java, adapte les chemins aux fichiers réellement attendus.

10.5 Tester volontairement un échec

Dans un dépôt d'exercice, remplace temporairement test -f README.md par test -f FICHIER_INEXISTANT.md. Pousse ce changement ou ouvre une PR : la CI doit échouer. Corrige ensuite la ligne et pousse la correction.

Pourquoi cet exercice est utile
Une CI qui réussit uniquement n'a pas encore prouvé qu'elle détecte les erreurs. Provoquer un échec contrôlé puis le corriger permet de vérifier que le mécanisme fonctionne réellement.

10.6 Lire les résultats

- Vert : toutes les étapes exécutées ont réussi.
- Rouge : une étape a échoué ; ouvrir les logs pour identifier la commande et l'erreur.
- En attente : le job n'a pas encore démarré ou attend une ressource.
- Une CI réussie ne garantit pas l'absence de bugs ; elle confirme uniquement que les contrôles configurés ont réussi.

11. Bonnes pratiques de travail

- Créer des commits petits, cohérents et faciles à comprendre.
- Écrire des messages de commit précis : docs:, feat:, fix:, ci:, refactor: peuvent aider à classer les changements.
- Exécuter git status avant les opérations importantes.
- Lire git diff et git diff --staged avant un commit.
- Créer une branche par fonctionnalité ou correction.
- Ouvrir une Pull Request avec un objectif clair et des vérifications indiquées.
- Ne pas committer de secrets, clés privées, mots de passe ou fichiers temporaires.
- Ne pas modifier l'historique d'une branche partagée sans coordination.
- Traiter les échecs CI avant de fusionner, selon les règles de l'équipe.
- Garder un README utile : objectif, prérequis, installation et commandes de démarrage.

12. Dépannage : erreurs fréquentes

Message / situation	Cause possible	Piste de résolution
fatal: not a git repository	Terminal hors du dépôt	cd vers le bon dossier puis git status
nothing to commit	Aucun changement sélectionné ou nouveau	Vérifier git status et les fichiers modifiés
rejected / non-fast-forward	Le distant contient des commits absents localement	Examiner l'état, puis intégrer le distant avant de pousser
src refspec main does not match any	Branche main inexistante ou aucun commit	Vérifier git branch et créer un premier commit si nécessaire
CONFLICT	Modifications incompatibles	Lire git status, résoudre les fichiers et terminer la fusion
CI rouge	Une commande du workflow a échoué	Lire les logs de l'étape précise et reproduire le contrôle localement
permission denied / authentication	Accès ou authentification incorrects	Vérifier le compte, le jeton ou la clé SSH et les droits du dépôt

Cas important : push rejeté

Ne réponds pas automatiquement à un rejet par git push --force. Commence par comprendre pourquoi les historiques divergent.

```
git status
git fetch origin
git log --oneline --graph --decorate --all
```

Selon l'état du dépôt et les règles de l'équipe, on pourra ensuite intégrer les changements distants avec merge ou rebase. Le choix doit être fait en comprenant les commits concernés.

13. Exercices pratiques et corrigés

Exercice 1 — créer un dépôt local

Objectif : créer un dépôt, ajouter un README et enregistrer deux commits.

15. Initialiser le dépôt sur la branche main.
16. Créer README.md avec un titre.
17. Ajouter le fichier et créer un premier commit.
18. Ajouter une section « Objectifs » et créer un second commit.
19. Afficher les deux commits avec git log --oneline.

Exercice 2 — travailler avec une branche

Objectif : isoler une modification.

20. Créer feature/ajouter-linux depuis main.
21. Ajouter une section Linux dans README.md.
22. Créer un commit puis pousser la branche.
23. Ouvrir une Pull Request vers main.
24. Après fusion, mettre main à jour localement et supprimer la branche si elle n'est plus utile.

Exercice 3 — simuler un conflit

Objectif : comprendre les marqueurs et résoudre un conflit.

25. Créer deux branches à partir du même commit.
26. Modifier la même ligne de README.md différemment dans chaque branche.
27. Fusionner une branche dans l'autre.
28. Résoudre les marqueurs, vérifier le fichier et terminer la fusion.

Exercice 4 — créer et tester une CI

Objectif : voir un workflow réussir puis échouer.

29. Créer .github/workflows/ci.yml avec le workflow de la section 10.
30. Pousser le fichier et vérifier l'exécution sur GitHub Actions.
31. Remplacer temporairement README.md par un nom de fichier absent.
32. Vérifier l'échec puis rétablir la bonne vérification.
33. Pousser la correction et confirmer le succès.

Corrigé indicatif — commandes principales

```
git init -b main
echo "# Mon projet" > README.md
git add README.md
git commit -m "docs: ajouter le README"
echo "\n## Objectifs\nApprendre Git et GitHub." >> README.md
git add README.md
git commit -m "docs: ajouter les objectifs"
git log --oneline
```

Les commandes de conflit et de Pull Request dépendent de l'état exact des branches et doivent être réalisées en observant les sorties de GitHub et de git status.

14. Fiche mémo des commandes

Besoin	Commande
Vérifier l'état	git status
Initialiser un dépôt	git init -b main
Cloner un dépôt	git clone URL
Créer une branche et s'y placer	git switch -c feature/nom
Changer de branche	git switch main
Sélectionner un fichier	git add fichier
Créer un commit	git commit -m "message clair"
Comparer le working tree	git diff
Comparer la staging area	git diff --staged
Afficher l'historique	git log --oneline --graph --all
Publier la branche la première fois	git push -u origin nom-branche
Publier les commits suivants	git push
Récupérer et intégrer les changements	git pull
Récupérer les références sans fusionner	git fetch
Voir les dépôts distants	git remote -v
Retirer de la sélection	git restore --staged fichier
Annuler un commit partagé	git revert identifiant

Mettre temporairement de côté	git stash push -m "description"
Lister les stash	git stash list
Voir les détails d'un commit	git show identifiant

Conclusion : ce que tu dois savoir faire

À la fin de ce parcours, tu dois être capable de créer un dépôt local, versionner proprement tes fichiers, relier le projet à GitHub, collaborer avec des branches et des Pull Requests, diagnostiquer les erreurs Git courantes et créer une CI simple avec GitHub Actions.

Lien avec la suite de la formation
Tu peux maintenant poursuivre Docker en gardant le dépôt devops-formations comme carnet de progression : documente chaque exercice, versionne les Dockerfiles et utilise une Pull Request pour les changements importants.